SQL Query Optimizer and Execution Engine

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / *Languages: C or C++*

1 Problem Statement

You will build a small SQL query engine that takes a SQL string, turns it into a query plan, applies one classical optimization to that plan, and then executes it against tables stored on disk. The result is an interactive shell where a user types a SQL query and gets back rows. The same shell can show the user the plan for a query both before and after optimization, side by side, so the speedup is visible.

You are not building a full database. You are building the part of a database that sits between the SQL the user types and the rows the user gets back. The storage layer is intentionally trivial: tables live as CSV files on disk and are read sequentially. There are no indexes, no transactions, no concurrency. Your effort goes into the four things this project is actually about: parsing a small subset of SQL, representing a query as a tree of operators, transforming that tree to make it faster, and executing the transformed tree row by row.

The single optimization you will implement is called *predicate pushdown*. It is the most important optimization in any SQL engine, and your benchmark will show that on a two-table query with a selective filter, your engine runs at least an order of magnitude faster with the optimizer enabled than without.

To keep the scope realistic for a semester project, the SQL subset is restricted to queries over **at most two tables at a time**. The benchmark data set contains three tables in total, and each query joins exactly two of them. Support for queries that reference three or more tables in a single `FROM` clause is available as a bonus (see Section 9).

2 What the Final Product Looks Like

When your project is complete, a user can compile your code with a single command and run an interactive shell that behaves like this:

```
$ make
$ ./sqlshell --data ./benchmark/data

sqlshell ready. Type \q to quit.

sql> CREATE TABLE users (id INT, name VARCHAR, country VARCHAR);
OK.

sql> CREATE TABLE orders (id INT, user_id INT, total DOUBLE);
OK.

sql> \load users users.csv
Loaded 5000 rows into users.

sql> \load orders orders.csv
Loaded 40000 rows into orders.

sql> \d
 Table    Rows
 users    5000
 orders   40000

sql> EXPLAIN
      SELECT users.name, orders.total
      FROM users, orders
      WHERE users.id = orders.user_id
        AND users.country = 'PK';

--- WITHOUT optimizer ---
Filter [users.id = orders.user_id AND users.country = 'PK']
```

```
  CrossProduct
    SeqScan users
    SeqScan orders

--- WITH optimizer ---
HashJoin [users.id = orders.user_id]
  Filter [users.country = 'PK']
    SeqScan users
  SeqScan orders

sql> EXPLAIN ANALYZE
      SELECT users.name, orders.total
      FROM users, orders
      WHERE users.id = orders.user_id
        AND users.country = 'PK';

--- WITHOUT optimizer: 4.12 seconds, 80 rows ---
--- WITH optimizer:    0.03 seconds, 80 rows ---
Speedup: 137.3x

sql> SELECT users.name, orders.total
      FROM users, orders
      WHERE users.id = orders.user_id
        AND users.country = 'PK';

 name              total
 Ayesha Khan       249.50
 Bilal Ahmed       108.00
 ...
(80 rows, 32 ms)

sql> \q
```

The two plans printed by `EXPLAIN` are the heart of this project. The version without the optimizer computes the full cross product of `users` and `orders` (10 billion intermediate rows) and only then filters. The optimized version pushes `users.country = 'PK'` down to filter `users` first, then turns the equality `users.id = orders.user_id` from a filter on the cross product into a hash join. The speedup is the entire point of the project.

3 Supported SQL Subset

Your engine must handle the following, and **nothing else**. Anything outside this subset may be rejected with an error message.

3.1 Data Definition

- `CREATE TABLE` *name* `(` *column declarations* `)` where each column is `INT`, `DOUBLE`, or `VARCHAR`. No constraints. No `NOT NULL`. No primary keys.

3.2 Query

- `SELECT` followed by a comma-separated list of qualified column references (`table.col`). No `*`, no expressions, no aliases, no aggregates.
- `FROM` followed by either a single table name or two comma-separated table names. No `JOIN` keyword; two-table queries always use the comma-separated form. No table aliases. Support for three or more tables is a bonus feature (Section 9).
- `WHERE` followed by a conjunction (`AND`-separated) of simple predicates. Each predicate is one of:
  - `table.col` *op* `table.col` (a join condition or a column-to-column filter)
  - `table.col` *op literal* (a filter against a constant)

  where *op* is one of `=`, `<`, `>`, `<=`, `>=`, `!=`.
- `LIMIT` *n*.

There is no `GROUP BY`, no `ORDER BY`, no `HAVING`, no `DISTINCT`, no `NULL`, no `OR`, no parentheses in the `WHERE` clause, no nested queries, no aggregate functions. The `WHERE` clause is always a flat list of

predicates joined by `AND`. This is by design: the smaller the surface area of the language, the more time you can spend on the parts that matter.

3.3 Diagnostic and Shell Commands

- `EXPLAIN` *query* — prints the query plan twice, once without the optimizer and once with it, side by side as shown in the example above.
- `EXPLAIN ANALYZE` *query* — runs the query both ways, prints the wall-clock time and row count for each, and reports the speedup ratio.
- `\d` — list loaded tables and their row counts.
- `\load` *table file.csv* — load rows from a CSV file into a previously created table.
- `\q` — quit.

4 Architectural Overview

Your system is organized as a pipeline of five stages. Each stage has a single, well-defined job. Keep them strictly separated; do not let parsing logic leak into the optimizer or execution logic leak into the parser.

```
SQL text
  |
  v
[ 1. Lexer ]      ---> token stream
  |
  v
[ 2. Parser ]     ---> query AST
  |
  v
[ 3. Planner ]    ---> logical plan tree (validated against catalog)
  |
  v
[ 4. Optimizer ]  ---> optimized plan tree (predicate pushdown applied)
  |
  v
[ 5. Executor ]   ---> rows
```

4.1 The Catalog

A simple in-memory map from table name to a list of (column name, column type) pairs, plus the row count once data is loaded. Used by the planner to validate column references and by the executor to interpret CSV rows. The catalog does not need to be persisted to disk; it is rebuilt at startup as the user runs `CREATE TABLE` and `\load`.

4.2 The Row Representation

A row is an array of values. A value is one of: a 64-bit integer, a double, or a string. Use a tagged union in C, or a `struct` with a type field, or a `std::variant` in C++. The same value type is used throughout the executor and the predicate evaluator.

5 Phase 1 — Parser and Plan Generation

The goal of Phase 1 is to take a SQL string and produce a validated logical plan tree. By the end of this phase, the `EXPLAIN` command must print a plan tree (the unoptimized version is enough; the optimized version arrives in Phase 2). Malformed queries must be rejected with an error message that mentions the line and column where the problem was found.

5.1 Lexer

Write a hand-written lexer. It reads characters from the SQL string and produces a sequence of tokens. The token kinds you need are:

- **Keywords**: `SELECT`, `FROM`, `WHERE`, `AND`, `LIMIT`, `CREATE`, `TABLE`, `INT`, `DOUBLE`, `VARCHAR`. Keywords are case-insensitive.
- **Identifiers**: a letter or underscore followed by letters, digits, or underscores. Used for table and column names.

- **Integer literals**: a sequence of digits.
- **Double literals**: digits with a decimal point.
- **String literals**: characters enclosed in single quotes. You do not need to handle escape sequences.
- **Operators**: `=`, `<`, `>`, `<=`, `>=`, `!=`.
- **Punctuation**: `,`, `(`, `)`, `.`, `;`.

Whitespace separates tokens and is otherwise discarded. The lexer needs to track the line and column of each token so the parser can produce useful error messages. The lexer is the simplest component in the project; expect it to be a few hundred lines of straightforward code.

5.2 Parser

Write a hand-written recursive-descent parser. Do not use a parser generator (Bison, Yacc, ANTLR, or any equivalent). The parser consumes the token stream from the lexer and produces an abstract syntax tree (AST).

You only need to parse two kinds of statements: `CREATE TABLE` and `SELECT`. The grammar is small enough that the parser is essentially a translation of the grammar into a function for each rule. A sketch of the grammar in EBNF:

```
statement       ::= create_stmt | select_stmt

create_stmt     ::= 'CREATE' 'TABLE' IDENT '(' col_decl (',' col_decl)* ')'
col_decl        ::= IDENT type
type            ::= 'INT' | 'DOUBLE' | 'VARCHAR'

select_stmt     ::= 'SELECT' col_ref (',' col_ref)*
                    'FROM' IDENT (',' IDENT)?
                    ('WHERE' predicate ('AND' predicate)*)?
                    ('LIMIT' INT_LIT)?

col_ref         ::= IDENT '.' IDENT
predicate       ::= operand op operand
operand         ::= col_ref | literal
literal         ::= INT_LIT | DOUBLE_LIT | STRING_LIT
op              ::= '=' | '<' | '>' | '<=' | '>=' | '!='
```

The AST node types are correspondingly small: `CreateTableStmt`, `SelectStmt`, `ColumnRef`, `Literal`, `Predicate`. Each node carries the source location of its first token for error reporting.

5.3 Validator and Plan Generator

After the parser produces an AST, walk the AST and check that it makes sense against the catalog. Specifically:

- Every table named in `FROM` must exist in the catalog. An unknown table is an error.
- Every `table.col` reference must name a column that exists in that table. An unknown column is an error.
- Every column referenced in `SELECT` or `WHERE` must use a table that appears in the `FROM` list. Referencing a column from a table not in scope is an error.
- In each predicate, the two operand types must be comparable. Comparing an integer to a string is an error. An integer compared to a double is allowed (the integer is promoted).

If validation passes, translate the AST into a logical plan tree. The logical plan node types you need are:

- **Scan**: leaf node, names a single table. Output is every row of that table.
- **CrossProduct**: two children. Output is every pairing of one row from the left child with one row from the right child.
- **Filter**: one child, holds a list of predicates joined by `AND`. Output is the rows from the child that satisfy every predicate.
- **Project**: one child, holds a list of column references. Output is each input row reshaped to contain

only those columns.

- **Limit**: one child, holds an integer. Output is the first n rows from the child.

The translation is mechanical:

1. If the `FROM` clause names one table, build a single `Scan` for it.
2. If the `FROM` clause names two tables t_1 and t_2, build `CrossProduct(Scan(`t_1`), Scan(`t_2`))`.
3. If the query has a `WHERE` clause, wrap the result in a `Filter` whose predicate list is the conjuncts of the `WHERE`.
4. Wrap that in a `Project` for the `SELECT` list.
5. If the query has a `LIMIT`, wrap the result in a `Limit`.

Because the `FROM` clause has at most two tables, the plan tree contains at most one `CrossProduct` node. This is an unoptimized plan, and it is intentionally bad for the two-table case. That is fine. Phase 2 will fix it.

5.4 The EXPLAIN Command

Implement a function that takes a plan tree and prints it as an indented text tree. Each node prints its type and its key parameters (the table name for a scan, the predicates for a filter, the column list for a project). At this stage, `EXPLAIN` prints only the unoptimized plan; in Phase 2 you will extend it to print the optimized plan as well.

6 Phase 2 — Predicate Pushdown

Phase 2 has one job: implement predicate pushdown. That is the entire optimizer. There is no statistics collection, no cost model, no join reordering, no search algorithm. You will write one tree-rewriting function and run it on the plan tree produced by Phase 1.

6.1 What Predicate Pushdown Does

A query like

```
SELECT users.name, orders.total
FROM users, orders
WHERE users.id = orders.user_id
  AND users.country = 'PK';
```

produces this plan from Phase 1:

```
Project [users.name, orders.total]
  Filter [users.id = orders.user_id AND users.country = 'PK']
    CrossProduct
      Scan users
      Scan orders
```

Predicate pushdown rewrites it into this:

```
Project [users.name, orders.total]
  HashJoin [users.id = orders.user_id]
    Filter [users.country = 'PK']
      Scan users
    Scan orders
```

Two things happened. First, the predicate `users.country = 'PK'` mentions only columns from `users`, so it was pushed down to filter the `users` scan directly, before the cross product. Second, the predicate `users.id = orders.user_id` is an equality between a column on the left side of the cross product and a column on the right side, so it was pulled into the cross product, turning the cross product into a hash join.

6.2 The Algorithm

You will write a single function that takes a plan tree and returns an optimized plan tree. Because the plan has at most one `CrossProduct` (two-table queries only), the function does not need to recurse. It walks the tree looking for the pattern `Filter` directly above `CrossProduct`. If the pattern is not present, the function returns the plan unchanged. If it is present:

1. Look at each predicate in the filter individually. For each predicate, determine which of the two tables below the cross product it mentions.
2. Sort the predicates into four buckets:
   - **Left-only**: mentions only columns from the left child's table.
   - **Right-only**: mentions only columns from the right child's table.
   - **Join condition**: an equality (`=`) with one side referencing the left table and the other side referencing the right table.
   - **Remainder**: anything else (for example, a non-equality comparison across the two tables).
3. Wrap the left `Scan` in a `Filter` containing the left-only predicates, if any. Do the same for the right `Scan` with the right-only predicates.
4. If at least one join condition was found, replace the `CrossProduct` with a `HashJoin` carrying that join condition. If there was more than one join condition, keep the first as the `HashJoin` condition and add the rest to the remainder bucket. If no join condition was found, keep the `CrossProduct`.
5. If the remainder bucket is empty, remove the original `Filter` above the join (or cross product) entirely. Otherwise keep the `Filter` with only the remainder predicates.

That is the entire optimizer. No statistics, no cost model, no search, no recursion. For a two-table query, these five steps produce the optimized plan directly.

6.3 Why This Is Worth Doing

The unoptimized plan computes the full cross product of the two tables before applying any filter. For tables of a few thousand and a few tens of thousands of rows, the cross product has tens or hundreds of millions of intermediate rows, which takes many seconds to materialize. The pushed-down version filters the left table down to perhaps a few hundred rows first, then runs a hash join that finishes in well under a second. The whole point of an optimizer is to make this kind of difference happen automatically, and predicate pushdown is the single most important rewrite that databases use to achieve it.

6.4 Extending EXPLAIN

Update the `EXPLAIN` command so that it prints *both* plans for the same query, labeled "WITHOUT optimizer" and "WITH optimizer." The user should see the difference at a glance. This side-by-side view is what your demo will show.

7 Phase 3 — Execution Engine and Benchmark

Phase 3 makes the plan trees actually run. You will build a simple executor in which each plan node type has a corresponding function that takes its input rows and returns its output rows. To execute a query, you walk the plan tree bottom-up: each node's executor receives the already-computed outputs of its children, does its job on them, and hands its own output back up.

7.1 Execution Model

There are two common ways to execute a query plan. The first is the *pipelined* model, in which rows flow up the tree one at a time and every operator is a small state machine that yields the next row on demand. This is what real databases do, and it is also hard to get right: each operator has to remember where it was between calls, and debugging a bug in the middle of a deep pipeline is unpleasant.

The second way is the *materialized* model. Each operator receives its complete input as a list (or array, or vector) of rows, does its entire job, and returns its complete output as another list of rows. There is no state held between calls because each call does all the work at once. This is the model you will use.

The tradeoff is memory: materialized execution holds the full intermediate results in RAM at every level of the plan. For a small data set this is fine. For the unoptimized version of your benchmark queries, however, the cross product of two tables becomes enormous, and your program will spend many seconds

(or run out of memory) trying to materialize it — which is precisely the point of the benchmark. The unoptimized version is *supposed* to be dramatically slow. Predicate pushdown makes the intermediate results small enough to fit comfortably in memory, which is exactly why the speedup is so large.

7.2 The Row and Value Types

A row is an array of values. A value is one of: a 64-bit integer, a double, or a string. Use a tagged union in C, or a `struct` with a type field, or a `std::variant` in C++. Comparison rules: integers compare as integers; doubles compare as doubles; strings compare lexicographically; an integer compared to a double promotes the integer. Comparing values of incompatible types is a runtime error and should abort the query with a clear message.

A table is a list of rows. In C++, use `std::vector<Row>`. In C, a dynamic array that grows as rows are appended works equally well.

7.3 The Executor Functions

Write one function per plan node type. Each function takes whatever inputs it needs and returns a list of rows. There are six of them, and none of them is longer than a screen of code.

7.3.1 Scan

```
vector<Row> exec_scan(string table_name);
```

Open the CSV file for the named table, read each line, parse it into a `Row` according to the column types from the catalog, and append it to a result vector. When the file ends, return the vector. That is the whole function.

7.3.2 Filter

```
vector<Row> exec_filter(vector<Row> input, vector<Predicate> preds);
```

Allocate an empty result vector. For each row in the input, evaluate every predicate against the row using the predicate evaluator (described below). If every predicate passes, append the row to the result. Return the result.

7.3.3 Project

```
vector<Row> exec_project(vector<Row> input, vector<ColumnRef> cols);
```

For each input row, construct a new row containing only the values at the requested column positions, in the requested order. Return the list of new rows.

7.3.4 CrossProduct

```
vector<Row> exec_cross_product(vector<Row> left, vector<Row> right);
```

Two nested loops. For each row ℓ in `left`, for each row r in `right`, concatenate ℓ and r into a combined row and append it to the result. Return the result. This is where the unoptimized plan spends all its time.

7.3.5 HashJoin

```
vector<Row> exec_hash_join(vector<Row> left, vector<Row> right,
                           JoinCondition cond);
```

Build a hash map from the left side: for each row in `left`, compute the value of the left join column, and insert the row into the map under that key. (Multiple rows may share the same key; use a map from key to a list of rows, or a multimap.) Then, for each row in `right`, look up the value of the right join column in the map; for every matching left row, concatenate and append to the result. Return the result.

7.3.6 Limit

```
vector<Row> exec_limit(vector<Row> input, int n);
```

Return a copy of the first `min(n, input.size())` rows of `input`. That is all.

7.4 The Execute Dispatcher

Glue the executor functions together with a single recursive function that takes a plan node and returns its output rows. In C++ this looks like:

```
vector<Row> execute(PlanNode* node) {
    switch (node->type) {
        case NODE_SCAN:
            return exec_scan(node->scan.table_name);
        case NODE_FILTER:
            return exec_filter(execute(node->filter.child),
                               node->filter.preds);
        case NODE_PROJECT:
            return exec_project(execute(node->project.child),
                                node->project.cols);
        case NODE_CROSS_PRODUCT:
            return exec_cross_product(execute(node->cross.left),
                                      execute(node->cross.right));
        case NODE_HASH_JOIN:
            return exec_hash_join(execute(node->join.left),
                                  execute(node->join.right),
                                  node->join.cond);
        case NODE_LIMIT:
            return exec_limit(execute(node->limit.child),
                              node->limit.n);
    }
}
```

Running a query end to end is then one call: `execute(optimized_plan)`. To measure `EXPLAIN ANALYZE` timings, wrap the `execute` call in a clock-reading pair.

7.5 The Predicate Evaluator

The filter and hash join functions both need to evaluate predicates. Write one function that takes a predicate (an AST node) and a row (or a pair of rows, for a join predicate), looks up the column values from the row, applies the operator, and returns true or false. This is a short function whose body is a switch statement over the six operators (`=`, `<`, `>`, `<=`, `>=`, `!=`). Use the comparison rules given in Section 7.2.

7.6 The SQL Shell

Build a REPL that reads a statement from standard input (terminated by a semicolon), runs it through the full pipeline, and prints the result. The shell supports:

- Multi-line input until a terminating semicolon.
- Tabular output with column headers and aligned columns.
- A row count and elapsed time printed after each query.
- The backslash commands listed in Section 3.3.
- `EXPLAIN` and `EXPLAIN ANALYZE`, both of which produce side-by-side output as shown in the example session.

For `EXPLAIN ANALYZE`, run the query twice: once by passing the unoptimized plan to `execute`, and once by passing the optimized plan to `execute`. Time each call with a clock. Print both times and the ratio.

7.7 Benchmark

The final deliverable is a benchmark that demonstrates your optimizer is doing real work. You will load a fixed data set consisting of three tables. The tables are sized so that unoptimized two-table queries are slow but finite — a few seconds to perhaps a minute — and optimized queries finish almost instantly.

You do not need to worry about running out of memory on the unoptimized runs; the provided data has been sized with this executor in mind.

You will run three queries against this data set. Each query joins exactly two of the three tables and has at least one predicate that is a column-to-literal comparison. The choice of which two tables each query uses is yours, as long as all three tables appear in at least one query across the suite.

For each query, your benchmark must print:

- The plan tree without the optimizer.
- The plan tree with the optimizer.
- The wall-clock execution time without the optimizer.
- The wall-clock execution time with the optimizer.
- The speedup ratio.

A successful submission shows speedups of at least **10x** on at least two of the three queries. The optimized version should be measurably and visibly faster, not by a few percent but by a large factor.

8 Deliverables and Submission

8.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation.
- A `Makefile` (or `CMakeLists.txt`) that builds the `sqlshell` binary with a single command. The build must succeed on a recent Ubuntu LTS using only the standard C/C++ toolchain.
- A `README.md` explaining how to build and run the project, what SQL features are supported, and any known limitations.
- A `grammar.txt` file containing the EBNF grammar of the SQL subset your parser actually accepts.
- A `design.pdf` document of at most 8 pages explaining how each of the five pipeline stages works in your implementation, the data structures you chose for the catalog and the operators, how predicate pushdown is implemented, and a brief discussion of the benchmark results.
- A `benchmark/` directory with the SQL files of your benchmark queries, the loaded data, a script that runs the benchmark suite, and the results table from your final run.
- A `tests/` directory with simple test cases for the lexer, the parser (including some malformed inputs that should produce errors), and the optimizer (showing example plans before and after pushdown).

8.2 Archive Naming

`Group[Number]_Project02_QueryOptimizer.zip`, for example `Group17_Project02_QueryOptimizer.zip`.

8.3 Demonstration

Each group will present a live demonstration during the week following the submission deadline. The demonstration consists of three parts. First, a brief walkthrough of how your code is organized using the design document. Second, a live execution of the benchmark suite where the instructor is free to type new queries on the spot. Third, a short oral examination in which each group member will be asked to explain a specific component of the implementation without notes. All group members must be present. Missing the demonstration results in a zero for Phase 3.

9 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any parser generator including Bison, Yacc, Lemon, ANTLR, or their equivalents. The parser must be hand-written recursive descent.
- Use of any existing SQL parser, query planner, or execution engine as a library. This includes but is not limited to DuckDB, SQLite, the `libpg_query` library, and any equivalent.
- Use of an ORM or query builder.
- Use of Python at any layer of the system, including test harnesses. Tests and benchmark scripts must be written in C, C++, shell, or JavaScript.

- Hardcoded query plans or query-specific shortcuts. The system must work on arbitrary queries within the supported subset.
- Submitting code whose commit history shows large, infrequent commits. Your git history should reflect the incremental development of a system of this size; histories with three commits totaling thousands of lines will be scrutinized for academic integrity.

10 Bonus Opportunities

Each of the following is independent and may be attempted in any combination. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points total. To claim bonus credit, your design document must include a dedicated section explaining which bonus items you attempted and how you verified them.

10.1 Three-Table Queries (up to 10%)

Extend the parser, planner, optimizer, and executor to support queries that reference three tables in the `FROM` clause. The grammar change is minor: allow `FROM` to list any number of tables. The planner change builds nested `CrossProduct` nodes left-to-right: `CrossProduct(CrossProduct(Scan(`t_1`), Scan(`t_2`)), Scan(`t_3`))`.

The optimizer change is where the real work is. Your pushdown function must now recurse into the children of the plan tree after processing the outer filter, so that predicates can also be pushed into the inner cross product. A predicate that mentions only t_3 needs to travel through two levels of cross product to land on `Scan(`t_3`)`. A predicate that mentions t_1 and t_3 is an equi-join condition for the outer cross product rather than the inner one. Thinking carefully about which predicates belong at which level is the point of this bonus.

The benchmark must include at least one three-table query and show its speedup, in addition to the required two-table queries.

10.2 Sort-Based ORDER BY (up to 5%)

Add `ORDER BY` support to the grammar, the planner, and the executor. The new operator sorts its input fully in memory (an in-memory sort is sufficient; no external sort required) before yielding rows. The benchmark must include at least one query using `ORDER BY` and a screenshot or printed output showing correctly ordered results.

The first item is short and required reading before you start. The remaining items are reference material to consult as you implement each phase.

1. Graefe, G. "Volcano — An Extensible and Parallel Query Evaluation System." *IEEE Transactions on Knowledge and Data Engineering*, 1994. The iterator model you will implement is described in this paper. You only need to understand Sections 1 through 3.
2. Pavlo, A. *CMU 15-445 Database Systems*, lectures 10 through 14 (Query Processing) and lecture 15 (Query Optimization). The video lectures are freely available on the CMU course website. They are the clearest available explanation of the material in this project.
3. Ramakrishnan, R. and Gehrke, J. *Database Management Systems*, 3rd edition. Chapter 12 covers query evaluation. Chapter 15 covers basic optimization including predicate pushdown.
4. Selinger, P. G. et al. "Access Path Selection in a Relational Database Management System." *ACM SIGMOD*, 1979. Optional. The foundational paper on cost-based optimization. Read it if you are curious about how real databases choose join orders, which is beyond the scope of this project.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday, 11:30 AM – 2:30 PM